Projet de programmation — Jeu d'exploration en monde infini

March 18, 2026

1 Introduction

De nombreux jeux vidéo modernes reposent sur des **mondes ouverts** dans lesquels le joueur peut se déplacer librement, explorer et interagir avec un environnement composé d'objets, de créatures ou de personnages. Dans certains jeux, ces univers donnent même l'impression d'être **potentiellement infinis** : de nouvelles zones apparaissent progressivement à mesure que le joueur explore le monde.

Par exemple, dans *Minecraft*, le monde est généré au fur et à mesure de l'exploration : lorsque le joueur se déplace, le programme crée automatiquement de nouveaux paysages. Lorsque le joueur ajoute ou enlève un bloc, son action persiste à jamais.

Derrière ces expériences se cachent plusieurs concepts fondamentaux en programmation : la représentation d'un monde complexe, la gestion d'entités multiples, la **génération procédurale** de contenu et la **persistance** des actions du joueur.

L'objectif de ce projet est de vous faire concevoir un **mini-jeu d'exploration reposant sur un monde potentiellement infini**. À travers ce projet, vous développerez des compétences essentielles en **C++**, notamment en programmation orientée objet, gestion de la mémoire, structures de données et sauvegarde de l'état du programme entre différentes exécutions.

L'objectif n'est pas de reproduire des jeux développés par des équipes entières pendant des années, mais de réaliser une **version simplifiée** reposant sur les mêmes principes fondamentaux.

Pour atteindre ces objectifs, vous réaliserez un **jeu vidéo** en **2D**, dans le terminal ou via une interface graphique, en **équipe de 4 étudiants**, répondant à plusieurs contraintes techniques qui structurent le projet et guident vos choix de conception. Nous vous encourageons à ne pas simplement implémenter un projet respectant les contraintes, mais bien à concevoir un jeu concret et captivant, issu de vos propres choix créatifs.

2 Objectifs pédagogiques

Ce projet vise à vous apprendre à maîtriser :

- la **programmation orientée objet** (hiérarchies de classes, héritage, polymorphisme, encapsulation) ;
- la **gestion de la mémoire** en C++ (allocation dynamique, destructeurs, etc.) ;
- la création et l'utilisation de **structures de données complexes** (notamment des matrices, grilles, ou structures de partitionnement d'un monde);
- la **gestion de fichiers** (compétence acquise en BA1), mais transposée en C++ pour sauvegarder et restaurer un état de jeu.
- la **gestion d'un projet** par la collaboration entre les membres du groupe et le découpage des tâches.

3 Contraintes

Votre jeu doit respecter les contraintes explicitées dans les points suivants. L'objectif est d'évaluer votre capacité à concevoir une solution propre, correcte et robuste en C++ répondant aux contraintes techniques. Votre créativité et votre prise de risque seront prises en compte dans l'évaluation. Ainsi, un jeu répondant aux critères mais étant **trop simpliste** n'obtiendra pas directement la moyenne.

3.1 Entités et hiérarchie de classes

Le joueur doit rencontrer des objets, des personnages, des créatures, des obstacles ou tout autre type d'entité. Ces entités doivent être représentées via une **structure hiérarchique de classes** démontrant votre maîtrise de l'OOP (héritage, polymorphisme, redéfinition de méthodes, etc.).

3.2 Affichage du jeu

Il est nécessaire de proposer une interface utilisateur permettant un affichage 2D du jeu. Celle-ci peut se limiter à une carte 2D affichée dans le terminal (stdout), utilisant des caractères UTF-8 pour représenter les différents éléments du jeu ainsi que le personnage. Vous êtes naturellement libres de proposer une implémentation plus avancée en vous appuyant sur une **bibliothèque graphique dédiée** (Qt par exemple) afin de créer une véritable interface graphique.

3.3 Monde infini

Le jeu doit se dérouler dans un "**monde infini**". Concrètement, un personnage contrôlable par le joueur doit pouvoir se déplacer sur une carte **sans jamais atteindre de limite**.

En pratique, le monde ne doit pas être entièrement stocké en mémoire ni préconstruit. Le joueur doit pouvoir explorer indéfiniment dans toutes les directions, sans rencontrer de bord ou de fin du monde.

Seules les zones nécessaires à un instant donné (par exemple, celles autour du joueur) doivent être maintenues en mémoire, tandis que les autres peuvent être régénérées ultérieurement si besoin.

3.4 Génération procédurale

Afin de satisfaire cette contrainte, il est nécessaire de mettre en place un mécanisme de **génération procédurale** permettant de créer le terrain ainsi que les différents objets ou personnages rencontrés.

Ce mécanisme ne peut pas simplement se baser sur un système de pioche aléatoire dans un ensemble de terrains préconçus. La génération doit être **pseudo-aléatoire** et **variée**, de sorte que différentes zones présentent des caractéristiques distinctes.

Enfin, cette génération doit être compatible avec la persistance : les éléments générés ou modifiés (terrain altéré, objets ramassés, entités déplacées, etc.) ne peuvent pas être perdus lorsque le joueur s'éloigne puis revient dans la zone.

3.5 Persistance en mémoire du monde

Les éléments rencontrés dans le monde doivent être **préservés** même si le joueur ne les a plus dans son champ de vision. En d'autres termes, le monde ne peut pas être simplement "redessiné" comme s'il était neuf : les modifications et découvertes doivent persister. Le joueur doit pouvoir **fermer complètement l'application** puis la relancer en retrouvant l'état précédent du jeu (position du joueur, entités, inventaire, etc.). Les zones explorées ou modifiées doivent être restaurées à l'identique ; les zones jamais visitées peuvent être régénérées de manière déterministe.

3.6 Contraintes de gameplay et d'exploration

Le jeu ne doit pas se limiter à une simple démonstration technique de génération de monde, mais proposer une **expérience de jeu minimale cohérente**.

À ce titre, les éléments suivants sont requis :

- **Progression du joueur** : le jeu doit inclure au moins un mécanisme de progression (acquisition d'objets, déblocage de zones, évolution du personnage, objectifs à atteindre, etc.), permettant de donner un sens à l'exploration.
- **Éléments du monde variés** : le monde doit contenir plusieurs types d'entités ou d'éléments interactifs, incluant au minimum :
  - un élément **infranchissable** (mur, eau, montagne, etc.) ;
  - un élément **interactif ou destructible** ;
  - au moins une **entité autonome ou semi-autonome** (PNJ, créature, ennemi, etc.).
- **Exploration motivée** : les interactions et la structure du monde doivent inciter le joueur à explorer (par exemple, via des ressources, des obstacles, des entités ou des objectifs).

Le jeu peut être réalisé en **vue du dessus** ou en **vue de profil**, à condition que ces choix restent compatibles avec les contraintes de monde infini, de génération procédurale et de persistance.

4 Pour aller plus loin

Les **bonus** vous permettent de distinguer votre projet en allant au-delà des contraintes obligatoires. Ils peuvent enrichir l'expérience de jeu, améliorer l'architecture logicielle ou démontrer une maîtrise plus avancée du C++. Les idées ci-dessous sont **purement indicatives** : vous êtes libres de proposer tout autre bonus pertinent.

4.1 Exemples de bonus

- **Interface graphique** via une bibliothèque (Qt, SFML, SDL, etc.).
- **Sauvegardes multiples** (plusieurs *slots*, chargement/suppression, infos de progression).
- **Multijoueur** local ou en réseau (Internet/LAN, client–serveur, synchronisation).
- **Joueur autonome / IA** (agents non joueurs, se mouvant grâce à un algorithme, ils ne doivent pas forcément être actifs si hors du champ de vision du joueur).
- **Intégration GenAI** (ex. via le plan gratuit de l'API Gemini) : dialogues PNJ, quêtes dynamiques, descriptions, assistant de jeu, etc. Le jeu doit rester fonctionnel sans API.
- **Extensions et idées ouvertes** : toute fonctionnalité supplémentaire pertinente pourra être proposée et implémentée, comme par exemple la génération procédurale avancée du monde, des systèmes de combat élaborés, du crafting (création d'outils à partir d'éléments collectés), de l'optimisation algorithmique ou toute autre mécanique originale.

5 Remise

Aucun rapport écrit n'est demandé. La soumission du projet consiste uniquement à déposer **trois fichiers** sur l'UV :

5.1 Diagramme UML

Un fichier contenant le **diagramme UML** du projet (au format **PDF** ou **image**). Ce diagramme doit représenter clairement :

- la hiérarchie de classes (héritage, polymorphisme) ;
- les relations principales entre classes (associations, compositions) ;
- les composants essentiels du jeu (monde, génération, entités, persistance, sauvegarde, etc.).

5.2 Vidéo de présentation

Une **vidéo** démontrant le fonctionnement du jeu, incluant idéalement :

- une présentation rapide de l'interface et du gameplay ;
- une démonstration du monde infini (dans la mesure du possible) et de la génération procédurale ;
- une preuve de la persistance (retour dans une zone déjà explorée / modifications conservées) ;
- une sauvegarde suivie d'une reprise de la partie (fermeture puis relance du jeu).

5.3 Code source

Un **fichier ZIP** contenant l'intégralité du code source (et des ressources éventuelles) nécessaire à la compilation et à l'exécution du projet. Ainsi qu'un fichier README.md[1] expliquant rapidement le projet ainsi que la démarche à suivre pour exécuter votre code.

Le code doit être :

- déposé sur l'UV sous forme de ZIP ;
- et *également à jour sur GitHub au moment de la soumission.*

N'oubliez pas de documenter votre code correctement !

6 Critères d'évaluation

Présentation orale et défense du projet La note finale est conditionnée à une **présentation orale en groupe**, suivie d'une défense du projet. Une présentation collective permettra d'exposer le jeu, les choix de conception et les principaux mécanismes implémentés. Elle sera suivie d'une session de questions-réponses avec les assistants, puis de questions individuelles.

Chaque étudiant doit être capable d'expliquer les choix techniques et conceptuels. Si certaines parties du code ne sont pas maîtrisées, la note pourra être **revue à la baisse**, même si le projet fonctionne.

Principe général d'évaluation L'évaluation porte à la fois sur le respect des contraintes techniques, la qualité de la conception et de l'implémentation, la cohérence et la créativité du jeu, ainsi que l'implication dans le travail de groupe. Une attention particulière sera accordée à la structure du code, à la clarté de l'architecture, à la qualité de la documentation et à la maîtrise des concepts utilisés.

Le projet contribue à hauteur de **4 points sur 20** à la note finale du cours.

Grille de notation

Critère	Poids
Conception orientée objet et diagramme UML	20%
Génération et monde infini	20%
Persistance et gameplay	20%
Jeu et créativité	20%
Travail de groupe (participation, GitHub)	20%

Bonus Des fonctionnalités supplémentaires (interface graphique, intelligence artificielle, multijoueur, systèmes de jeu avancés, etc.) permettent d'obtenir des points bonus. Un projet incluant des bonus pertinents peut atteindre jusqu'à **5/20** pour la note du projet (+1 point à la note totale du cours), ce qui permet de compenser d'éventuelles pertes de points dans d'autres parties du cours.

[1] `https://docs.github.com/fr/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax`

Documentation La documentation est attendue uniquement pour les fonctions non triviales : une description claire de l'objectif, des paramètres et de la valeur de retour est requise, tandis que les commentaires internes doivent se limiter aux parties complexes ou non évidentes.

Exemple de documentation attendue

```java
/**
 * @brief Génère une tuile du monde à partir de ses coordonnées globales.
 *
 * Cette fonction utilise une génération pseudo-aléatoire déterministe afin
 * de garantir que la même position produise toujours le même terrain.
 *
 * @param x Coordonnée X dans le monde.
 * @param y Coordonnée Y dans le monde.
 * @return Tile La tuile générée (terrain, obstacles, etc.).
 */
Tile generateTile(int x, int y) {
    // Une ligne très complexe qu'il faut absolument expliquer dans le code
        directement.
    int seed = x * 73856093 ^ y * 19349663;
    ...
}
```

Travail de groupe Le projet doit être développé via **GitHub**. Chaque membre du groupe est attendu à contribuer activement au projet, à travers des commits réguliers. Le travail doit donc être relativement équitable.

7 Usage de l'IA générative

L'utilisation de l'IA générative (ChatGPT, Copilot, etc.) est autorisée pour vous aider à :

- comprendre des concepts liés au code ou non ;
- réaliser des recherches sur des problèmes techniques ;
- vérifier la grammaire et l'orthographe de votre documentation ;
- générer des idées créatives (noms, univers, objets, compétences, etc.) ;
- générer les graphiques en UTF-8, ou dans le cas où vous utiliseriez une bibliothèque graphique, les images que vous utiliserez.

Toutefois, **vous ne pouvez pas utiliser l'I.A. pour générer du code source ou le texte de votre documentation.** Le code doit être entièrement issu de votre travail personnel. Vous devez être capable d'expliquer et de justifier chaque partie du code que vous soumettez. Vous êtes responsable de la qualité et de l'originalité du code soumis. Toute tentative de plagiat ou de soumission de code non original sera sanctionnée conformément aux règles de l'université.